Functional Specification: LAN Change Sharing Tool

Working name: `changeshare` **Version:** 0.1 (draft) **Author:** Ayush **Status:** Draft for review

1. Overview

1.1 Purpose

Developers often need to hand uncommitted work (local changes or a stash) to a colleague before it is ready for a commit, branch or pull request. Today this is done by copy-pasting code. `changeshare` is a Linux command-line tool that sends uncommitted changes directly from one developer's machine to another on the same office network, with no commit, no branch and no author information attached.

1.2 Goals

- Send all working-tree changes or a stash to a teammate in one command.
- The receiver applies the changes as uncommitted edits, so anything they later commit is under their own name.
- Transfers stay inside the office network. No cloud service is involved.
- Setup takes under two minutes and needs no accounts.

1.3 Non-goals

- Not a replacement for git, pull requests or code review.
- No cloud relay or remote-teammate support in v1.
- No Windows or macOS support in v1.
- No real-time collaboration (use a live-share tool for that).
- No persistent storage of transferred changes on any server.

2. Constraints and Design Decisions

ID	Decision	Reason
C1	**Transfers are LAN-only.** There is no cloud relay.	Code never passes through a server run by the developer or a third party, which is far easier to defend under company policy.
C2	The tool lives in a **personal repository**, never in the company codebase.	It is a separate developer tool, not company software.
C3	Target platform is **Linux** only.	All office machines run Linux.
C4	Transfers are **peer to peer**: sender hosts, receiver pulls.	No central infrastructure to build or maintain.
C5	Only a **patch** is sent, never commit history.	Guarantees no author, committer or commit metadata is shared.

3. Users and Environment

3.1 User roles

- **Sender:** a developer with uncommitted changes in a git repository.
- **Receiver:** a colleague with a copy of the same repository, on the same or a close base commit.

3.2 Runtime environment

- Linux (desktop distributions in use at the office).
- Node.js 18 or later (or a single compiled binary with no Node needed).
- Git 2.30 or later.
- Both machines on the same local network with device-to-device traffic allowed.
- `avahi-daemon` for automatic discovery; a manual IP fallback is provided when it is missing.

4. Functional Requirements

4.1 Selecting what to send

ID	Requirement
FR-1	The sender can send **all current changes**, including tracked modifications, deletions and new untracked files. Gitignored files are excluded.
FR-2	The sender can send a **stash**: the latest (`stash`) or a specific one (`stash@{n}`).
FR-3	The sender can send **selected files** only, by passing paths.
FR-4	The sender's own staged changes, index and working tree must not be modified by sending.
FR-5	Binary files are included in the patch.
FR-6	If there are no changes, the tool prints "No changes to share" and exits without starting a transfer.

4.2 Preview and safety checks (sender side)

ID	Requirement
FR-7	Before sending, the tool shows a **summary**: file list, lines added and removed, and total size.
FR-8	The user can open a full diff preview.
FR-9	The tool **scans the patch for secrets**: API keys, tokens, private keys, passwords and `.env` -style files. Findings are shown and the user must explicitly confirm or remove the files.
FR-10	Files matching a configurable denylist (default: `.env*` , `*.pem` , `*.key` , `id_rsa*`) are excluded unless the user overrides with a flag.

ID	Requirement
FR-11	The user must confirm before the transfer starts (skippable with `--yes`).

4.3 Sharing and discovery

ID	Requirement
FR-12	The tool generates a **short, human-readable one-time code** (for example `blue-tiger-42`).
FR-13	The sender's machine starts a temporary local server on a configurable port (default `8765`) and announces itself on the LAN through **mDNS**, tagged with a hash of the code.
FR-14	The receiver enters the code and the tool **discovers the sender automatically**.
FR-15	If discovery fails, the sender sees its LAN IP and port, and the receiver can connect with `--host <ip>`.
FR-16	The code is **single-use** and **expires** after a configurable time (default 10 minutes). The server shuts down after one successful delivery, on expiry, or on cancel.
FR-17	Only the local network interface is used. The server never binds to a public interface or opens external connections.

4.4 Receiving and applying

ID	Requirement
FR-18	The receiver runs `changeshare receive <code>` inside their repository.

ID	Requirement
FR-19	The sender sees a **connection request** (receiver hostname and IP) and must approve before any data is sent.
FR-20	The receiver sees a summary (files, size) and confirms before applying.
FR-21	The tool checks the receiver's **base commit** against the sender's. If they differ, it warns and offers a three-way apply.
FR-22	The patch is applied to the working tree as **uncommitted changes**. Nothing is committed and no author metadata is created.
FR-23	The tool runs a dry-run check first. On conflict, it offers three-way apply or writes `.rej` files, and lists the affected files.
FR-24	If the receiver's working tree has local changes, the tool warns and offers to stash them first.
FR-25	The tool records an **undo point** so the receiver can revert the applied changes with `changeshare undo`.

4.5 Anonymity

ID	Requirement
FR-26	The payload contains only the diff and the minimum metadata needed to apply it (base commit hash, file list, tool version). It contains **no** author name, email, commit messages, branch names or file ownership.
FR-27	The sender's username and hostname are shown to the receiver only on the live connection prompt, never stored in the patch.
FR-28	The tool does not send telemetry.

5. Command-Line Interface

5.1 Commands

Command	Description
`changeshare send [paths...]`	Share working-tree changes (all, or only the given paths).
`changeshare send --stash [ref]`	Share the latest stash, or a given stash ref.
`changeshare receive <code>`	Connect to a sender by code and apply the changes.
`changeshare undo`	Revert the most recent applied transfer.
`changeshare doctor`	Check git, Node, avahi, firewall and port, and report problems.
`changeshare --version` / `--help`	Version and usage.

5.2 Options

Option	Applies to	Description
`--yes`	send, receive	Skip confirmation prompts (secret findings still require explicit confirmation).
`--port <n>`	send	Port for the temporary server.
`--expires <min>`	send	Code lifetime in minutes.
`--host <ip>`	receive	Connect directly, skipping discovery.
`--include-ignored-denylist`	send	Override the default secret-file denylist.
`--3way`	receive	Force a three-way apply.
`--dry-run`	receive	Show what would change without applying.

5.2 Example session

```
Sender:    $ changeshare send
           3 files changed, +42 -7 (2.1 KB). No secrets found. Send? [Y/n] y
           Share code: blue-tiger-42   (expires in 10 min)
           Waiting for receiver...
           Rahul (192.168.1.34) wants to receive. Allow? [y/N] y
           Delivered. Done.

Receiver:  $ changeshare receive blue-tiger-42
           Found sender on LAN. 3 files, +42 -7. Apply? [Y/n] y
           Applied as uncommitted changes. Run `changeshare undo` to revert.
```

6. Workflows

6.1 Send and receive (happy path)

1. Sender runs `send`. The tool builds the patch from a temporary index so the sender's own staging is untouched.
2. The tool runs the secret scan and shows the summary. The sender confirms.
3. The tool generates a code, starts the temporary server and announces on mDNS.

4. Receiver runs `receive <code>`. The tool discovers the sender and requests a connection.
5. Sender approves the request.
6. Sender and receiver establish an encrypted channel using the code (section 7).
7. The patch is delivered and the sender's server shuts down.
8. The receiver sees a summary, confirms, and the tool applies the patch and saves an undo point.

6.2 Conflict handling

1. The dry-run check fails.
2. The tool lists conflicting files and offers: three-way apply, abort, or apply the clean files only.
3. After a three-way apply, conflict markers are left in files for the receiver to resolve, exactly as in a normal merge.

6.3 Discovery failure

1. mDNS discovery times out (default 10 seconds).
2. The receiver sees: "Couldn't find the sender automatically. Ask them for their IP shown on their screen, then run `changeshare receive <code> --host <ip>`."
3. If the direct connection also fails, the tool suggests running `changeshare doctor`.

7. Security Requirements

ID	Requirement
SR-1	**LAN-only by design.** No cloud relay, no external network calls, no accounts.
SR-2	The transfer is **encrypted end to end**. The key is derived from the share code using a password-authenticated key exchange (PAKE, for example SPAKE2) so a short code cannot be brute-forced offline from captured traffic.
SR-3	The code is **single-use**, expires, and the server accepts only one successful session.

ID	Requirement
SR-4	After a small number of failed code attempts (default 3), the server shuts down and a new code is required.
SR-5	The **sender must approve** every receiver before data is sent.
SR-6	**Secret scanning** runs before every send (FR-9, FR-10).
SR-7	Patches are **never written to disk** on the sender except in memory, and on the receiver only to a temporary file that is deleted after apply. The undo point stores only the reverse patch, in the repo's `.git` directory.
SR-8	The receiver applies the patch with git's own `apply`, never by executing anything from the payload. Paths are validated so the patch cannot write outside the repository.
SR-9	**No telemetry.** Logs are local and contain no code contents.

8. Payload Format

A single message containing:

- `version`: tool/protocol version.
- `baseCommit`: commit hash the patch is based on.
- `files`: list of changed paths with status (added, modified, deleted, renamed).
- `patch`: the git binary-safe unified diff.

The payload is compressed, then encrypted. It contains no author, email, branch, hostname or commit message fields.

9. Error Handling

All errors show a plain-language cause and a next step, with no stack traces unless `--debug` is set.

Situation	Message and behaviour
Not inside a git repository	"This folder isn't a git repository. Run from inside your project."
Port already in use	Try the next free port automatically and show the new one.
Firewall blocks the port	"The receiver couldn't connect. Allow the port with `sudo ufw allow <port>/tcp` or run `changeshare doctor`."
mDNS unavailable	Fall back to the manual IP flow (FR-15).
Wrong or expired code	"Code not recognised or expired. Ask the sender for a new one."
Different base commit	Warn and offer three-way apply (FR-21).
Patch does not apply	Conflict flow (section 6.2).
Sender cancels or closes	Receiver is told the transfer was cancelled and nothing was changed.
Client isolation on the network	`doctor` detects a failed peer connection and explains that the network blocks device-to-device traffic and IT must allow it.

10. `doctor` Checks

- Git installed and version supported.
- Node version (if not using the binary).
- Current folder is a git repository.
- `avahi-daemon` running (mDNS available).
- Default port free, and whether the firewall (`ufw` or `firewalld`) allows it.
- Local LAN IP detected.

- Optional peer test: connect to another machine running `doctor --listen` to confirm device-to-device traffic works.

Each check reports pass, warn or fail with the fix.

11. Non-Functional Requirements

ID	Requirement
NFR-1	**Performance:** a typical patch (under 5 MB) transfers in under 3 seconds on office wifi. Patches up to 100 MB are supported; above that the tool warns.
NFR-2	**Usability:** a new user can send and receive using only the two-command quick start, with no configuration.
NFR-3	**Reliability:** the sender's repository is never modified; a failed or cancelled transfer leaves both repositories unchanged.
NFR-4	**Compatibility:** runs on the Linux distributions and git versions used at the office.
NFR-5	**Footprint:** starts in under 1 second and uses no resources after the transfer ends.
NFR-6	**Maintainability:** one core module with the CLI as a thin layer, so a VS Code extension can reuse it later.

12. Technical Approach

- **Language:** TypeScript on Node.js.
- **Git operations:** `simple-git`, or direct `git` calls for the temporary-index and apply steps.
- **LAN discovery:** `bonjour-service` (mDNS).
- **Transfer:** a short-lived HTTP or WebSocket server on the sender; the receiver connects as a client.
- **Encryption:** a PAKE library for the key exchange, then AES-256-GCM for the payload.

- **Secret scanning:** pattern-based scanner (regex rules for common key formats plus entropy checks), with the denylist.
- **Packaging:** internal use via the personal private repository (`npm i -g github:<user>/changeshare`) or a single binary built with `bun build --compile`.

13. Distribution and Rollout

- Hosted in a **personal private repository**, never in the company codebase.
- Shared with colleagues as a private-repo install or a compiled binary sent directly.
- README with install, the two-command quick start, and a troubleshooting section for firewall and mDNS.
- Rollout: test with one colleague, then 2-3 more for a week, then fix issues and share with the wider team.

14. Acceptance Criteria

1. Two Linux machines on the office wifi can complete a send and receive in under 30 seconds, using only the code.
2. The applied changes appear as uncommitted edits, and a commit made by the receiver shows only the receiver's name.
3. The patch contains no author, email, branch or commit message data.
4. The sender's staged changes and working tree are identical before and after sending.
5. A file containing a fake API key triggers a warning before sending.
6. A code cannot be used twice and stops working after expiry.
7. A receiver without sender approval receives no data.
8. `undo` restores the receiver's working tree to its state before applying.
9. With mDNS disabled, the manual `--host` flow works.
10. With the port blocked, `doctor` identifies the cause and the fix.
11. A conflicting patch triggers the conflict flow and never leaves the repository in a half-applied state.
12. No network traffic leaves the local network during any operation.

15. Milestones

Milestone	Scope	Estimate
M1: Core	Patch building, one-time code, LAN transfer, apply on receiver	1 week
M2: Safety	Preview, secret scanning, sender approval, expiry, encryption	1 week
M3: Polish	`doctor`, undo, conflict handling, clear errors, README	1 week
M4: Rollout	Pilot with colleagues, fixes, binary build	1 week
Later	VS Code extension wrapping the core	Optional

16. Risks and Open Questions

Item	Detail
Network isolation	The office wifi may block device-to-device traffic. This must be tested first with two laptops. If blocked, IT would need to allow it, since the LAN-only rule rules out a relay.
Company policy	Confirm with the team lead or IT that using an unapproved tool to move code between machines is acceptable.
Ownership	Build the tool on personal time and equipment, and check the employment agreement on side projects.
Different base commits	How strict should the tool be when base commits differ: warn only, or block? (Current proposal: warn and offer three-way.)
Distribution	Private repository install or compiled binary? Decide after the first pilot.

Item	Detail
Secret scanner depth	Start with regex rules and a denylist, or integrate an existing scanner?